

Reviewer Pack — Minimal Local Indefinite Integral and Communication Complexi...

Entry 7e3ee8bbbac5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 11:16:09 UTC

Minimal Local Indefinite Integral and Communication Complexity Rank Correlation via Graph Laplacians

Entry ID: 7e3ee8bbbac5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 11:16:09 UTC

1. Conjecture

Field A (mathematical branch): Graph Laplacian Spectral Geometry **Field B** (complexity object): Communication Complexity

Statement:

For every n -vertex communication complexity function f , the ratio of the smallest non-zero eigenvalue of the normalized graph Laplacian associated with the interaction graph of f to the number of vertices n is at least $\Omega(\log n)$. Equivalently, if the communication complexity rank $r(f)$ of f is defined as the smallest vertex degree in a corresponding graph, then $\log(r(f)) = \Theta(\log n)$.

Rationale (proposer's reasoning):

Graph Laplacian spectral geometry has been successfully applied to various areas of mathematics and physics. It has shown potential in capturing structural information from graphs. Communication complexity, which measures the amount of communication needed to solve a problem between two parties, can be related to graph theory. This conjecture suggests that the spectral properties of the interaction graph, as captured by the graph Laplacian, are directly linked to the communication complexity rank of the function.

Taxonomy category: Graph_Laplacian_Spectral_Geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a2db09947b4b90fd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every communication complexity function f , if the ratio of the smallest non-zero eigenvalue $\lambda_{\min}(L_G)$ of the normalized graph Laplacian L_G to the number of vertices n is $\leq 1/2 * \log(n)$, the conjecture is falsified. If this ratio is $\geq 1/2 * \log(n)$, the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Graph Laplacian" AND "Communication Complexity" AND "spectral geometry" - "normalized Laplacian eigenvalues" IN communication complexity - "rank correlation" IN graph laplacians AND communication complexity

Top relevant hits considered: - [<http://arxiv.org/abs/2303.13145v3>] Normalized Laplacian eigenvalues of hypergraphs - [<http://arxiv.org/abs/1611.05311v2>] On graphs with three or four distinct normalized Laplacian eigenvalues - [<http://arxiv.org/abs/1506.05762v1>] Remarks on Bounds of Normalized Laplacian Eigenvalues of Graphs - [<http://arxiv.org/abs/2008.05665v1>] Graph Complexity and Link Colorings - [<http://arxiv.org/abs/2311.15289v2>] Counting cliques without generalized theta graphs - [<http://arxiv.org/abs/1408.5939v2>] Planar Induced Subgraphs of Sparse Graphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        G = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    G[i][j] = G[j][i] = 1
        return G

    def degree(G, v):
        return sum(G[v][u] for u in range(len(G)))

    def communication_complexity_rank(G):
        return min(degree(G, v) for v in range(len(G)))

    def adjacency_matrix_to_laplacian(A):
        n = len(A)
```

```

D = [[0] * n for _ in range(n)]
for i in range(n):
    row_sum = sum(A[i][j] for j in range(n))
    D[i][i] = row_sum
L = [[D[i][j] - A[i][j] if i == j else -A[i][j] for j in range(n)] for i in range(n)]
return L

def normalize_matrix(M):
    n = len(M)
    sum_elements = sum(sum(row) for row in M)
    return [[M[i][j] / sum_elements for j in range(n)] for i in range(n)]

def smallest_non_zero_eigenvalue(L_norm):
    n = len(L_norm)
    if n == 0:
        return 0
    eigenvalues = []
    A = L_norm[:]
    while A:
        v = [A[i][i] for i in range(n)]
        norm_v = sum(v[i] * v[i] for i in range(n)) ** 0.5
        if norm_v == 0:
            break
        q = [v[i] / norm_v for i in range(n)]
        A = [[A[i][j] - q[i] * q[j] for j in range(n)] for i in range(n)]
        eigenvalues.append(norm_v)
    return min(eigenvalues) if eigenvalues else 0

def qr_decomposition(A):
    n = len(A)
    Q = [[Fraction(0, 1)] * n for _ in range(n)]
    R = [[Fraction(0, 1)] * n for _ in range(n)]
    for k in range(n):
        v = [A[k][i] for i in range(k, n)]
        norm_v = sum(v[i] * v[i] for i in range(len(v))) ** 0.5
        Q[k][k] = Fraction(norm_v).limit_denominator()
        R[0][k] = A[k][k]
        for j in range(k + 1, n):
            q = [Q[j][i] / Q[k][k] for i in range(k, n)]
            R[j][k] = sum(q[i] * v[i] for i in range(len(v)))
            for i in range(k, n):
                A[j][i] -= q[i] * v[i]
    return Q, R

def matrix_multiplication(A, B):
    n = len(A)
    C = [[Fraction(0, 1)] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            C[i][j] = sum(A[i][k] * B[k][j] for k in range(n))
    return C

def transpose_matrix(M):
    n = len(M)
    M_t = [[Fraction(0, 1)] * n for _ in range(n)]
    for i in range(n):

```

```

        for j in range(n):
            M_t[j][i] = M[i][j]
    return M_t

def determinant(matrix):
    if len(matrix) == 2:
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    det = Fraction(0, 1)
    for j in range(len(matrix)):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += (-1) ** j * matrix[0][j] * determinant(submatrix)
    return det

def inverse_matrix(M):
    n = len(M)
    det_M = determinant(M)
    if det_M == 0:
        raise ValueError("Matrix is singular and does not have an inverse.")
    cofactors = [[Fraction(0, 1)] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[:i] + matrix[i+1:]]
            cofactors[i][j] = (-1) ** (i + j) * determinant(submatrix)
    adjugate = transpose_matrix(cofactors)
    inv_M = [[adjugate[i][j] / det_M for j in range(n)] for i in range(n)]
    return inv_M

def solve_linear_system(A, b):
    n = len(A)
    A_augmented = [A[i] + [b[i]] for i in range(n)]
    Q, R = qr_decomposition(A_augmented)
    x = [Fraction(0, 1)] * n
    for j in range(n - 1, -1, -1):
        x[j] = (Q[n-1][j] - sum(Q[i][j] * x[i] for i in range(j + 1, n))) / R[j][j]
    return x

def generate_random_communication_complexity_function(n):
    G = generate_random_graph(n)
    r_f = communication_complexity_rank(G)
    f = [random.randint(0, 1) for _ in range(r_f)]
    return G, f

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    G, f = generate_random_communication_complexity_function(n)
    L_norm = normalize_matrix(adjacency_matrix_to_laplacian(G))
    lambda_min = smallest_non_zero_eigenvalue(L_norm)
    r_f = communication_complexity_rank(G)

    if lambda_min == 0 or n == 1:
        continue

    ratio = lambda_min / n
    results.append({

```

```

        "n": n,
        "lambda_min": lambda_min,
        "r_f": r_f,
        "ratio": ratio
    })

metric_value = sum(result["ratio"] for result in results) / len(results)
instances_tested = len(results)
n_max = max(result["n"] for result in results)
conjecture_holds = all(result["ratio"] >= 0.5 * math.log(result["n"]) for result in results)
counterexample = "" if conjecture_holds else "lambda_min/n < 0.5 * log(n)"

return {
    "metric_name": "Ratio of smallest non-zero eigenvalue to n",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = (sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results)) ** 0.5
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"lambda_min/n < 0.5 * log(n)\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3863038.py", line 177, in <module>
    trial_result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3863038.py", line 140, in run_trial
    L_norm = normalize_matrix(adjacency_matrix_to_laplacian(G))
    ~~~~~^~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3863038.py", line 47, in normalize_mat
    return [[M[i][j] / sum_elements for j in range(n)] for i in range(n)]
    ~~~~~^~~~~~
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure it does not affect the validity of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14704
2	propose	ollama_remote	glm4:latest	0	0	12462
3	propose	ollama_remote	glm4:latest	0	0	12409
4	propose	ollama_remote	glm4:latest	0	0	22881
5	preregistration	ollama_remote	glm4:latest	0	0	9279
6	novelty	ollama_remote	glm4:latest	0	0	8327
7	novelty	ollama_remote	glm4:latest	0	0	10930
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20091
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15871
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17205
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27968
12	judge	ollama_remote	glm4:latest	0	0	17236

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 189363 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/7e3ee8bbbac5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7e3ee8bbbac5.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7e3ee8bbbac5.tar.gz (if generated)